Bhagyesh Patil **D20A 03**

Blockchain Experiment 8

AIM: Implementing a Blockchain Platform using Ganache

Theory:

1. Blockchain Technology

A blockchain is a distributed, decentralized, and immutable ledger that records transactions across a network of computers. Each record (called a block) contains a cryptographic hash of the previous block, a timestamp, and transaction data, forming a chain. Because every participant holds a copy of the ledger and any alteration to a block invalidates all subsequent blocks, the data stored on a blockchain is highly tamper-resistant and transparent.

Key properties of blockchain:

- Decentralization: No single entity controls the network; authority is distributed across nodes. • Immutability: Once data is written to the blockchain, it cannot be altered or deleted. • Transparency: All transactions are visible to network participants.
- Consensus: Nodes agree on the state of the ledger through consensus mechanisms (e.g., Proof of Work, Proof of Stake).
- Security: Cryptographic hashing (SHA-256) ensures data integrity.

2. Ethereum & Smart Contracts

Ethereum is an open-source, decentralized blockchain platform that supports programmable transactions through Smart Contracts. A smart contract is a self-executing program stored on the blockchain whose terms are written directly into code. Once deployed, a smart contract runs automatically when predefined conditions are met, without the need for intermediaries.

Smart contracts in Ethereum are written in Solidity, a statically-typed, contract-oriented programming language. They are compiled into bytecode and deployed to the Ethereum Virtual Machine (EVM), which executes the contract logic on every node in the network.

Key concepts:

- Gas: A unit measuring the computational effort required to execute operations. Users pay gas fees in ETH to incentivize miners/validators.
- ABI (Application Binary Interface): Defines how to interact with the deployed contract — the function signatures and data encodings.
- Address: Every deployed contract gets a unique Ethereum address on the blockchain.
- msg.sender: A global variable in Solidity that holds the address of the account that called the current function.

3. Ganache

Ganache (by Truffle Suite / Consensys) is a personal blockchain for Ethereum development. It simulates a full Ethereum node locally on your machine, providing:

- 10 pre-funded test accounts, each loaded with 100 ETH.
- Instant mining — transactions are confirmed immediately (no wait time).
- A local RPC server (default: http://127.0.0.1:7545) that tools like MetaMask and Remix can connect to.
- A GUI showing real-time accounts, blocks, transactions, and logs for easy inspection.

Ganache is used exclusively for development and testing. It allows developers to deploy and test contracts without spending real ETH or waiting for mainnet/testnet confirmations.

4. MetaMask

MetaMask is a cryptocurrency wallet and browser extension that acts as a bridge between a web browser and the Ethereum blockchain. It allows users to manage Ethereum accounts, sign transactions, and interact with decentralized applications (dApps) directly from the browser. In development, MetaMask is configured to point to a local Ganache network instead of the Ethereum mainnet, enabling safe testing with dummy ETH.

5. Remix IDE

Remix IDE is a web-based integrated development environment for Solidity smart contract development, available at remix.ethereum.org. It provides a code editor, Solidity compiler, contract deployer, and debugger in a single interface. Remix can connect to various environments including the built-in JavaScript VM, Hardhat, or an Injected Provider (MetaMask), the last of which routes deployment through MetaMask to whichever network (Ganache, testnet, or mainnet) MetaMask is currently connected to.

6. Supply Chain Use Case

A supply chain smart contract automates the tracking of goods as they move between parties — in this experiment, a pharmaceutical product (medicine) passing through four stages: Manufacturer, Distributor, Retailer, and Customer. Each stage updates the contract state on-chain, creating a transparent and tamper-proof audit trail. This eliminates the need for a trusted central authority to verify ownership transfers and ensures all parties see the same data.

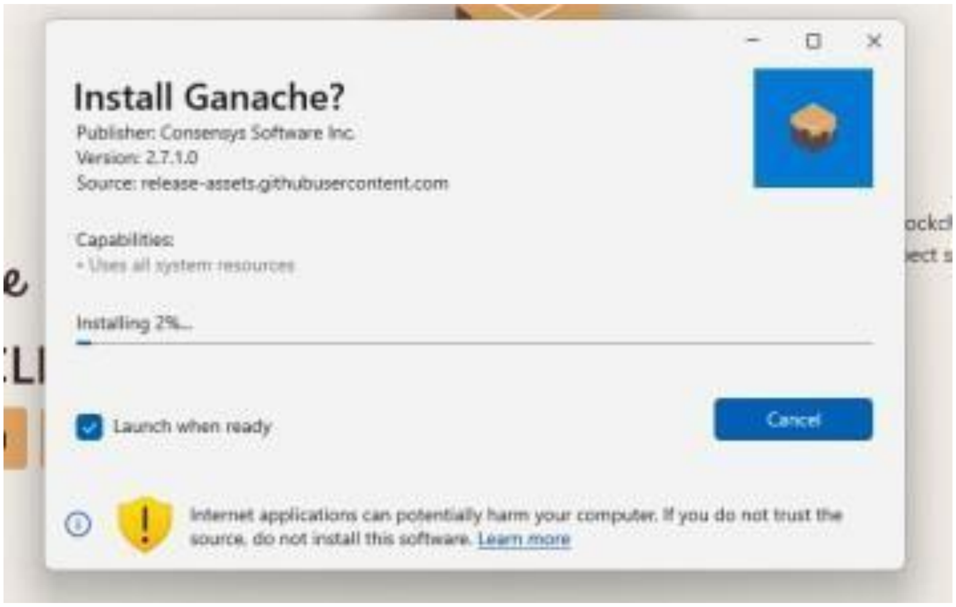
Implementation:

Step 1: Install Ganache

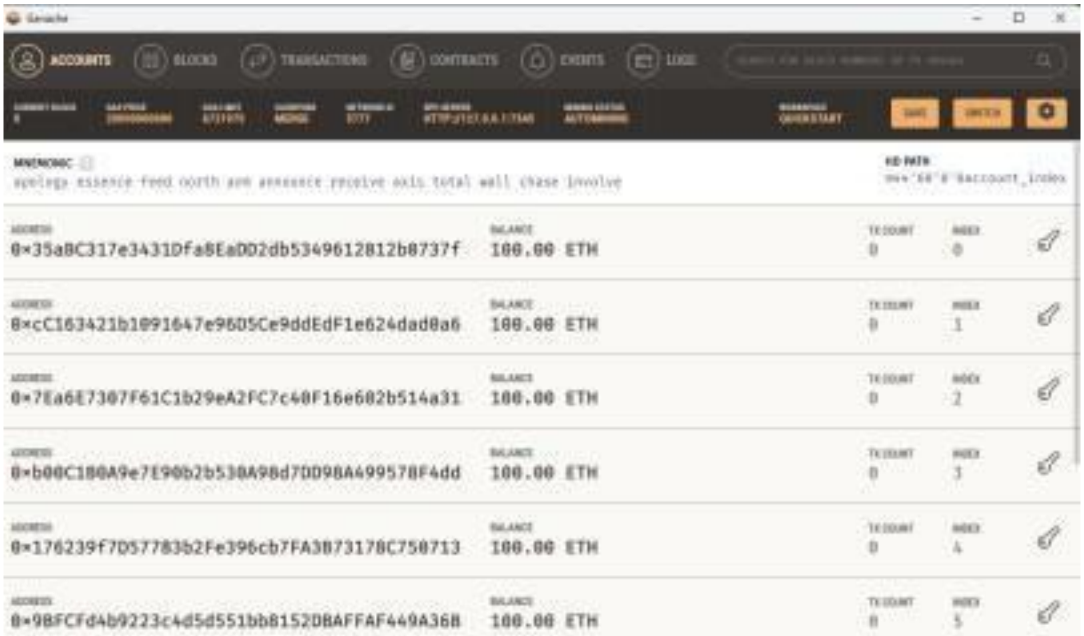
Ganache is a personal blockchain for Ethereum development used to deploy contracts, develop applications, and run tests.

1. Download Ganache v2.7.1 from the official Truffle Suite website (release assets.githubusercontent.com).

2. Run the installer. The installation prompt shows publisher as Consensys Software Inc.
3. Check 'Launch when ready' and click Install.



4. Once installed, open Ganache and click Quickstart Ethereum to start a local blockchain.



Note the following details from Ganache for later use:

- RPC Server URL: http://127.0.0.1:7545
- Network ID: 5777
- Private keys of accounts (click the key icon next to each address)

Step 2: Install & Setup MetaMask

MetaMask is a browser extension that serves as an Ethereum wallet and allows interaction with decentralized applications.

2.1 Install MetaMask

5. Open Google Chrome and go to the Chrome Web Store.
6. Search for MetaMask (by Consensys Software Inc.) and click Add to Chrome.

7. After installation, click 'Create a new wallet' on the MetaMask welcome screen.

8. Set a password and save the Secret Recovery Phrase securely.

2.2 Add Ganache Network to MetaMask

9. In MetaMask, click the network dropdown and select 'Add a custom network'.

10. Enter the following details for the RPC

URL: RPC URL: http://127.0.0.1:7545

RPC Name: Ganache

11. Click 'Add URL'.

12. Fill in the remaining custom network details:

• Network Name: Localhost 8545

• Default RPC URL: Ganache (127.0.0.1:7545)

• Chain ID: 1337

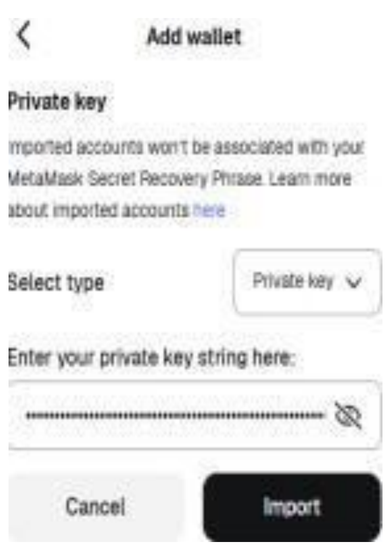
• Currency Symbol: ETH

13. Click Save.

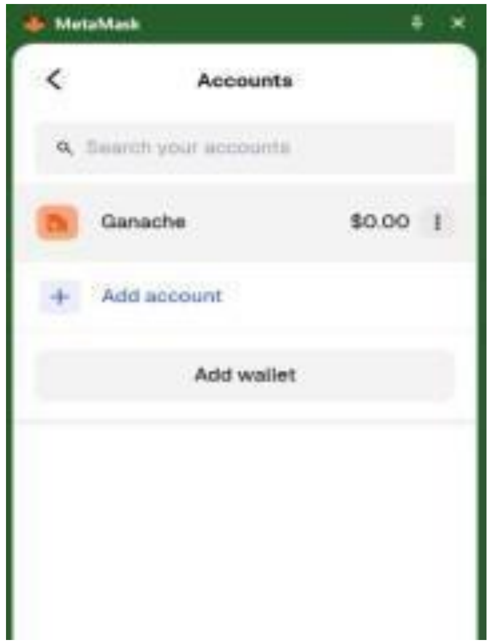
2.3 Import Ganache Accounts into MetaMask

14. In Ganache, click the key icon next to an account address to reveal its private key.

15. In MetaMask, go to Accounts > Add wallet > Import using Private Key. 16. Paste the private key and click Import.



17. The imported account (labeled 'Ganache') now appears in MetaMask under the Localhost 8545 network.



Step 3: Create the Smart Contract in Remix IDE

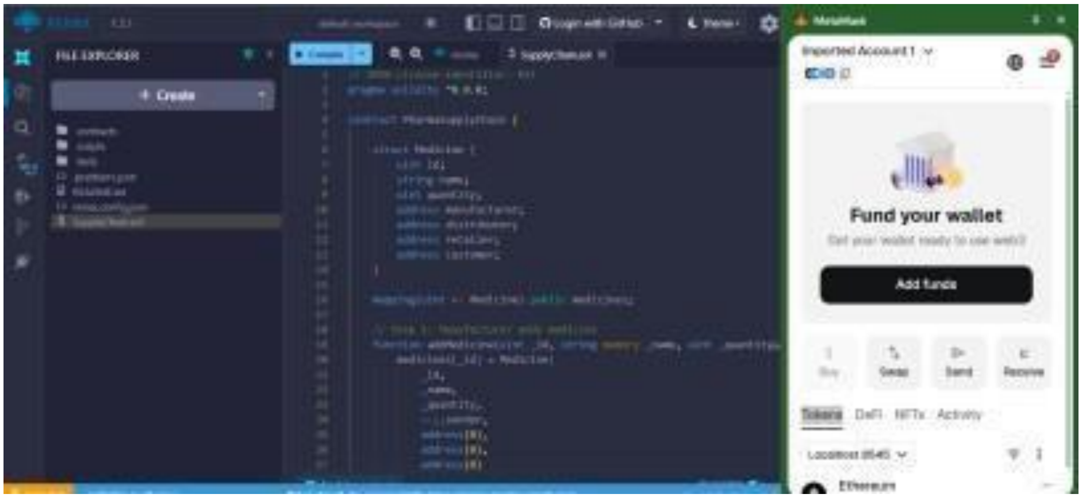
Remix IDE is a web-based Solidity development environment at remix.ethereum.org.

18. Open remix.ethereum.org in the browser.
19. In the File Explorer, create a new file named VotingSystem.sol.
20. Write the following smart contract for a Voting System chain:

5777

Step 4: Compile the Smart Contract

21. In Remix, click the Solidity Compiler icon on the left sidebar.
22. Select compiler version 0.8.x.
23. Click Compile SupplyChain.sol.
24. A green checkmark confirms successful compilation.



Step 5: Connect Remix IDE with MetaMask

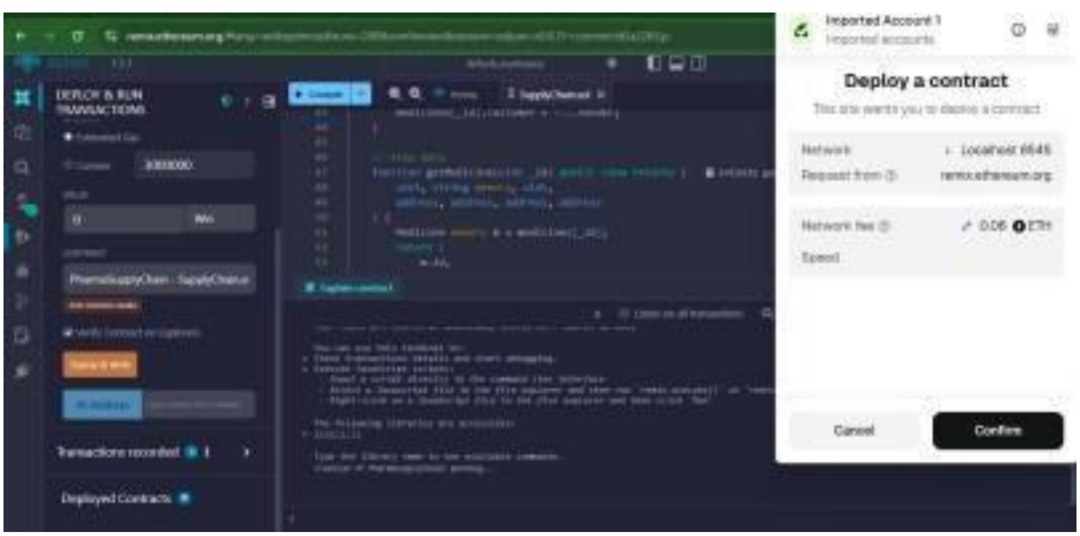
25. In Remix, navigate to the Deploy & Run Transactions tab.

26. In the Environment dropdown, select Injected Provider - MetaMask. 27. MetaMask will prompt a connection request — click Connect.

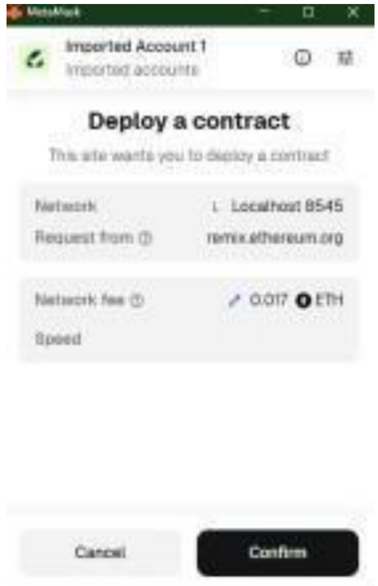
28. Ensure MetaMask is switched to the Localhost 8545 (Ganache) network. 29. The imported Ganache account will now appear as the active account in Remix.

Step 6: Deploy the Smart Contract

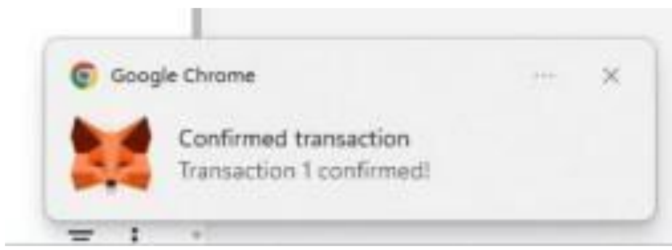
30. In the Deploy & Run panel, confirm VoteSystem is selected under Contract. 31. Click Deploy. MetaMask will open a confirmation popup.



32. Review the Network fee and click Confirm in MetaMask.

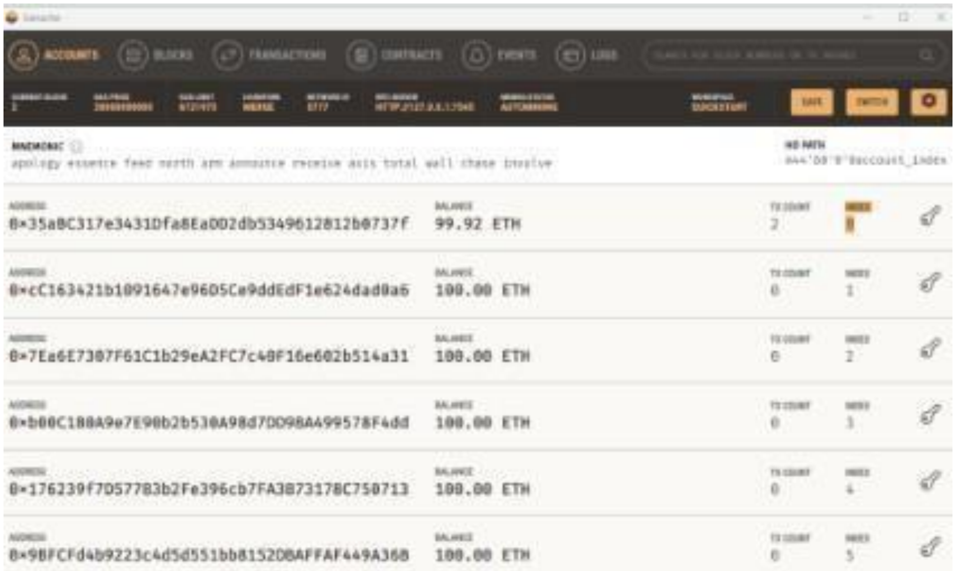


33. A 'Confirmed transaction' notification from Chrome confirms the deployment was successful.



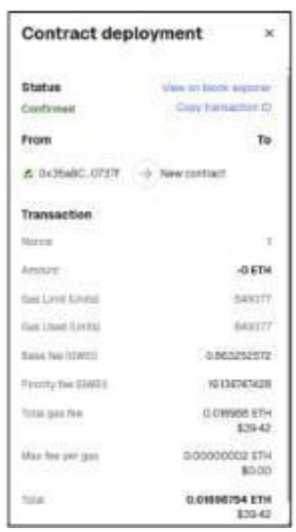
Step 7: Verify Transaction in Ganache

After deployment, Ganache reflects the transaction. The deploying account's balance decreases due to gas fees paid.



The contract deployment details in MetaMask show:

- Status: Confirmed
- From: 0x35aBC...0737f → To: New contract
- Gas Used: 849,377 units
- Total gas fee: 0.016988 ETH
- Account balance after: 99.92 ETH



Step 8: Interact with the Smart Contract

After deployment, the contract functions appear in the Remix Deploy & Run panel under Deployed Contracts.

Transaction Flow — Pharma Supply Chain

Account 1 (Manufacturer) — Call addMedicine:

```
addMedicine(1, "Paracetamol", 100)
```

Account 2 (Distributor) — Switch account in MetaMask, then call:

```
sendToDistributor(1)
```

Account 3 (Retailer) — Switch account in MetaMask, then call:

```
sendToRetailer(1)
```

Account 4 (Customer) — Switch account in MetaMask, then call:

```
purchase(1)
```

Call getMedicine to verify the full supply chain:

```
getMedicine(1)
```

Expected output from getMedicine(1):

- id: 1
- name: Paracetamol
- quantity: 100
- manufacturer: 0x35aBC...0737f (Account 1 address)
- distributor: 0xcC163...d0a6 (Account 2 address)
- retailer: 0x7Ea6E...a31 (Account 3 address)
- customer: 0xb00C1...4dd (Account 4 address)

Output

- Medicine batch created successfully on the Ganache local blockchain.
- Complete supply chain flow executed: Manufacturer → Distributor → Retailer → Customer.
- All transactions confirmed and visible in Ganache with updated block counts and reduced ETH balances.
- Smart contract deployed with status Confirmed; gas fees deducted correctly.

Conclusion

The experiment was successfully carried out by implementing the Ganache platform and integrating it with MetaMask and Remix IDE. The smart contract was compiled, deployed, and executed using Ganache accounts, which provided a local blockchain environment with pre-funded accounts.

The transactions were processed instantly, and all details such as gas usage, block creation, and account balances were observed directly in Ganache. This provided a clear understanding of how blockchain transactions work in a controlled environment without requiring real cryptocurrency.

Thus, the experiment helped in understanding the practical implementation of a local blockchain, wallet integration, and smart contract deployment successfully.